Kazakhstan Practical Driving Exam Simulator

Category B - Astana Autodrome

Damir Tassybayev
tassybayev.kostanay@gmail.com

Course: Interactive Graphics
Instructor: Prof. Marco Schaerf
Sapienza University of Rome

Contents

1 Introduction

This project is a browser-based 3D simulation of the **practical driving examination of the Republic of Kazakhstan** for the category-B licence. The player drives a car around a faithful recreation of an Astana *autodrome* (driving-test ground) and is graded, in real time, by an automated examiner that reproduces the official penalty system: turn-signal checkpoints, a hill start (*estakada*), traffic lights, boundary lines, and two timed parking exercises. The simulator was developed as the final project for the Interactive Graphics course at Sapienza University of Rome.

1.1 Graphics environment

The application runs in the browser through WebGL, using **Three.js** (release r160) as the rendering layer and **tween.js** for eased interpolation. The renderer is a `WebGLRenderer` with antialiasing enabled, the `sRGB` output colour space, soft shadows (`PCFSoftShadowMap`), and a device pixel ratio capped at 2 to avoid over-drawing on high-DPI screens.

A deliberate design choice is that the project has **no build step**. All modules are native ES modules resolved through an `importmap` in `index.html`, and every third-party library is vendored locally under `lib/`. This keeps the repository self-contained and lets it run directly from GitHub Pages (or any static file server) without bundling.

1.2 Running the simulator

Because the project uses ES modules, it must be served over HTTP rather than opened from the file system. Any static server works, for example:

```
python -m http.server 8000     # from the repository root
# then open http://localhost:8000
```

Listing 1: Running the simulator locally.

It has been tested on Chromium-based browsers and Firefox on Windows. Because the controls are keyboard-driven, mobile devices are not supported.

2 User Manual

2.1 Premise and objective

The candidate starts the car at the START box at the top of the autodrome and drives a fixed route through a sequence of exercises. The automated examiner accumulates *penalty points* for mistakes. The examination is **passed** if, and only if, the candidate completes the course and the cumulative penalty stays *below 100 points*; reaching 100 points, whether through one critical error or several smaller ones, ends the examination immediately as a failure. Penalties fall into three severity tiers, summarised in Table 1.

Table 1: Penalty severity tiers.

Points	Severity	Examples
100	Critical (instant fail)	Crossing on a red/yellow light; not finishing a parking in time
25	Major violation	Missing a full stop; heavy hill rollback
5	Minor mistake	Missing a turn signal at a checkpoint
20	Boundary touch	A wheel on a penalty (boundary) line

2.2 Controls

The car is driven with the keyboard; the mouse orbits the camera only in the free-orbit view. The full scheme is listed in Table 2.

Table 2: Controls.

Input	Action
`W` / `S` or ↑ / ↓	Accelerate / brake and reverse
`A` / `D` or ← / →	Steer left / right
`Space`	Full-stop handbrake (toggle)
`Q` / `E`	Left / right turn signal (toggle)
`H`	Hazard lights (toggle)
`C`	Cycle the camera view
`L`	Toggle the headlights
`O`	Open / close the doors
`R`	Reset the car and the examination
Mouse drag / wheel	Orbit / zoom (in the *orbit* camera only)

2.3 Camera modes

The `C` key cycles four views, handled by the `CameraManager`. The default *fixed* view is rigidly bolted to the car: it translates and rotates with the body so the car always sits in the same place on screen. *Cockpit* places the camera at the driver's seat looking forward. *Top* looks straight down, useful for the parking exercises. *Orbit* frees the camera with `OrbitControls` so the player can drag and zoom around the car. The non-orbit modes are positioned each frame from a car-local offset converted to world space, so they follow the body exactly.

2.4 HUD overview

Two panels overlay the scene. The top-left panel reports the live vehicle state: speed in km/h, gear (D, R or P when the handbrake is engaged), the active camera, the headlight state, and the current turn signal. The top-right *examiner* panel shows the running penalty total, colour-coded from green to amber to red as it climbs, together with the most recent violation. When the examination ends, a full-screen overlay announces PASSED or FAILED with the final penalty total and, on failure, the reason.

3 The Car: hierarchical model

The car is the project's complex hierarchical model and is built entirely in code in `world/car.js`; nothing about it is imported, in line with the course rule that animations may not be imported.

3.1 Part hierarchy and assembly

The car is a tree of `THREE.Group` nodes. The root node carries the whole car and is the only node the driving model moves; every child therefore follows automatically. The hierarchy is:

- **root** — position and heading set by the driving model;
  - **body** — lower body, cabin, glass band, chrome bumpers, seats; the emissive head-, brake- and indicator-light lenses; the two hinged *doors*; and the *steering wheel* (a

torus nested in two helper groups so it can spin about its own axle regardless of its tilt);

- **front-left** and **front-right pivots** — each a group that yaws for steering and contains a wheel that spins for rolling;
- **rear-left** and **rear-right wheels** — spin only;
- **two headlight spot lights** aimed forward.

The car's local forward direction is $+X$ and up is $+Y$. Key dimensions are a half-wheelbase of 1.45 m, a half-track of 0.82 m, a wheel radius of 0.42 m and a steering lock of `MAX_STEER` = 0.52 rad ($\approx 30°$). Because the front wheels are children of their steering pivots, they turn in place and still roll correctly; this two-level joint/visual split is what lets every joint rotate independently without disturbing the others.

3.2 Materials and lights

The body uses `MeshStandardMaterial` instances (metallic paint, dark glass, chrome bumpers, rubber tyres). The light lenses use emissive materials whose intensity is animated: the headlight lenses and two non-shadowing `SpotLight`s fade in when the lights are toggled, the brake lenses brighten while braking or reversing, and the amber indicator lenses blink at about 2 Hz. Car-local *left* is the $-Z$ side and *right* is $+Z$, so the left/right indicators light the correct side of the body.

4 The Autodrome (environment)

4.1 Map image and pixel-to-world mapping

The driving ground is a single photographic image of a real Astana autodrome, laid flat on a textured plane (`world/examGround.js`). The image (1237×848 px) is loaded asynchronously with `TextureLoader` and the ground plane is sized to the *same aspect ratio*, so a pixel on the image maps exactly to a point on the ground. The plane is oriented so that the image top is world north ($-Z$) and the image right is world east ($+X$).

Every element of the examination is positioned by reading its location directly off the image in pixels and converting it with the helper in `world/mapCoords.js`:

```
export function px2world(px, py) {
  return {
    x: (px / IMG_W - 0.5) * AUTO_W,   // 0..IMG_W  (left->right)  -> world X
    z: (py / IMG_H - 0.5) * AUTO_H,   // 0..IMG_H  (top->bottom)  -> world Z
  };
}
```

Listing 2: Pixel-to-world mapping (`mapCoords.js`).

This single source of truth keeps "what is painted on the image" and "what the car interacts with" in agreement: the car start, the hill, and all examination lines are expressed in image pixels and converted through `px2world`.

4.2 The overpass (estakada)

The hill-start exercise is a real raised structure built in `world/overpass.js`. Its footprint is taken from image pixels (it spans image $x = 550$ on the east, where the west-bound car arrives, to $x = 210$ on the west). The deck is an extruded side profile: an up-ramp, a flat crest carrying a painted STOP line and a STOP sign on the right edge, and a down-ramp. A height function returns the deck elevation at any point, and zero elsewhere:

```
function heightAt(x, z) {
  if (z < z0 || z > z1 || x > xA || x < xD) return 0;
  if (x >= xB) return H * (xA - x) / (xA - xB); // up ramp
  if (x <= xC) return H * (x - xD) / (xC - xD); // down ramp
  return H;                                     // crest
}
```

Listing 3: Overpass deck height (`overpass.js`).

The driving model samples this function so the car physically rides up, over and down the deck (Section 5).

4.3 Course markings and traffic lights

The examination lines that must be visible are drawn on the ground by `world/courseMarks.js`: the boundary "penalty" lines as thin black strips, and a red stop line at each traffic-light location. At each traffic light a small hierarchical light model (post, housing and three lamps) is placed and driven by a green $\rightarrow$ yellow $\rightarrow$ red cycle, desynchronised between lights. The remaining rule lines (turn-signal, full-stop, parking, finish) are invisible triggers by default to keep the photographic map clean; a debug flag draws them all for verification.

4.4 Lighting and shadows

The scene is lit by a `HemisphereLight` ambient term, so no surface is fully black, and a `DirectionalLight` acting as the sun, which casts the scene shadows through a 2048×2048 shadow map. The car's headlights add two forward spot lights at night. Shadows use percentage-closer filtering for soft edges, and the renderer works in the sRGB colour space for correct tone.

5 Driving model

5.1 Kinematic bicycle model

The car is driven by a kinematic *bicycle model* (`vehicle.js`); no physics engine is used, keeping the motion light and predictable. The state is the planar position (x, z), the heading ψ, the signed speed v, and the current steering angle δ. Throttle accelerates, braking decelerates and then engages reverse, and releasing both applies rolling friction toward zero. With wheelbase $L = 2.9\,\mathrm{m}$ the pose integrates as

$$\dot{\psi} = \frac{v}{L}\tan\delta, \qquad \dot{x} = v\cos\psi, \qquad \dot{z} = -\,v\sin\psi.$$

Speed is clamped to $[-5, 15]$ m/s (≈ -18 to 54 km/h) and the steering lock is reduced at higher speed for stability. The travelled distance is fed back to the wheels so the rolling animation always matches the real speed.

```
if (Math.abs(this.speed) > 1e-3) {
  this.heading += (this.speed / WHEELBASE) * Math.tan(this.steer) * dt;
}
this._dir.set(Math.cos(this.heading), 0, -Math.sin(this.heading));
const dist = this.speed * dt;
this.x += this._dir.x * dist;
this.z += this._dir.z * dist;
```

Listing 4: Pose integration (`vehicle.js`).

5.2 Riding the terrain

When the car is over the overpass, the model raises it to the deck height and pitches it to the local slope. The pitch is taken from two height samples a short distance e ahead of and behind the car along its travel direction, and a component of gravity along the slope is added to the speed, so the hill must be actively held:

$$\theta_{\text{pitch}} = \operatorname{atan2}(h_f - h_b,\ 2e), \qquad \dot{v} \mathrel{+}= -\,g\,\sin\theta_{\text{pitch}}.$$

The resulting position and orientation are pushed to the car root as a quaternion (yaw then pitch), so the body sits correctly on the ramp.

5.3 Handbrake and stall

Pressing `Space` toggles a full-stop handbrake: the speed is forced to zero and throttle is ignored until it is released, with the brake lights held on. The model also detects a stall when throttle and brake are held together near a standstill, which is reported to the examiner.

6 Animations

All animation is computed in JavaScript every frame; nothing is imported.

- **Wheels.** All four wheels spin about their axle by an angle that advances with the travelled distance, $\Delta\theta = \dfrac{\mathrm{d}s}{r}$, where r is the wheel radius. The two front wheels additionally yaw with the steering pivots.
- **Steering wheel.** The cabin steering wheel rotates about its own axle proportionally to the steering angle.
- **Doors.** Each door is a hinge group; opening and closing is driven by **tween.js** with a `Quadratic.Out` easing for a smooth swing.
- **Lights.** Headlight and brake intensities ease toward their target; the turn indicators blink; the brake lenses respond to braking and reversing.
- **Traffic lights.** Each light advances through its green/yellow/red cycle and updates the emissive lamp intensities.

7 The examiner: scoring system

The examination logic is a generic, data-driven engine that reads a course definition and applies a rule per element each frame, accumulating penalties.

7.1 Scoring engine

The `Scoring` class (`exam/scoring.js`) keeps a running total and an ordered event log, and fails the examination as soon as a critical (100-point) penalty fires or the total reaches the threshold:

```
penalize(exercise, rule, points, t = 0) {
  if (this.isOver) return;
  this.total += points;
  this.log.push({ t, exercise, rule, points });
  this.lastViolation = this.log[this.log.length - 1];
  if (points >= FAIL_THRESHOLD || this.total >= FAIL_THRESHOLD) {
    this.status = 'failed';
```

```
    this.failReason = rule;
  }
}
```

Listing 5: Penalty accumulation (`scoring.js`).

7.2 Data-driven course

The whole route lives in one editable configuration file, `exam/course.js`, with every position expressed in image pixels. It groups the elements by type: *signal lines* (a crossing requires a given indicator), *light lines* (a crossing on red/yellow fails), a *stop line* (a full stop is required before crossing), *penalty lines* (a wheel on the line costs points), two timed *parkings*, and the *finish line*. The engine (`exam/exam.js`) converts each pixel segment to a world segment once at start-up and then evaluates it generically.

7.3 Line-crossing detection

The car is reduced to a reference point (its front axle) and its four wheels in world space. A line is the segment $A \to B$; the side of the car relative to the line is the sign of the 2-D cross product

$$s = (B_x - A_x)(P_z - A_z) - (B_z - A_z)(P_x - A_x).$$

A crossing is registered when s flips sign between frames *and* the projection of P onto the segment lies within its extent. Each rule then reacts: a signal line checks the active indicator, a light line reads its traffic signal, the stop line checks that a one-second stop was registered nearby, and the finish line ends the examination. The penalty (boundary) lines instead use a point-to-segment distance against each wheel, with a short cooldown so a wheel resting on a line is charged once rather than every frame.

7.4 Parking and hill exercises

Each parking exercise arms when the car crosses its trigger line and then gives a fixed time budget (30 s). It succeeds when the required wheels (the two rear wheels for the 90° box, the two right wheels for parallel parking) are held stationary inside the target line for one second; if the budget expires first, the examination fails. The hill exercise is scored against the overpass footprint: rolling back more than 30 cm from the furthest-forward point, or stopping more than 0.5 m from the STOP line (or not stopping at all), each costs a major penalty.

8 Resource attribution

Libraries (not developed by the team).

- **Three.js** (MIT) — `https://threejs.org/`; release r160, used for rendering, the scene graph and the `OrbitControls` addon.
- **tween.js** (MIT) — `https://github.com/tweenjs/tween.js/`; eased interpolation for the door animation.

Both libraries are vendored under `lib/` so the repository is self-contained.

Assets. The autodrome ground is a single photographic top-down image of an Astana driving-test ground (`assets/textures/autodrome.png`). All other geometry (the car, the overpass, the traffic lights, the course markings) and all animation are generated in code by the team; no 3D models or animation clips are imported.

Code. All application code under `src/` was written by the team. The procedural asphalt textures are generated at run time on an HTML canvas.